

LifeButler 智能生活管家最终报告

课程：《程序设计实习》

项目名称：LifeButler 智能生活管家

小组成员：师小川、何维昊、黄涵优

GitHub 仓库：<https://github.com/shilinchuan/LifeButler>

最终 Release：<https://github.com/shilinchuan/LifeButler/releases/tag/v0.3.0-final>

生成日期：2026 年 6 月 25 日

一、项目概述

LifeButler 是一个本地运行的桌面生活管理系统，使用 PyQt6 构建界面，SQLite 保存本地数据，并通过 matplotlib 绘制可视化图表。项目目标是把日常生活中的任务、目标、记账、健康、备忘录和复盘流程整合到同一个应用中，形成“计划、执行、记录、反馈”的闭环。

本项目不是单一功能的小工具，而是围绕个人生活管理场景设计的综合应用。用户可以在同一个界面内完成快速记录、今日任务安排、专注执行、目标推进、健康打卡、周报复盘和数据备份。系统不接入云端服务，也不依赖外部 AI API，所有数据默认保存在用户本机，适合课程项目演示和个人本地使用。

二、需求背景与设计目标

日常生活管理经常存在三个问题：

1. 信息分散。任务、账单、健康记录、备忘录往往分散在多个应用中，用户难以在同一视角下判断当天状态。
2. 执行脱节。普通待办列表只能记录事项，但很难把任务与目标、周计划和专注执行联系起来。
3. 反馈不足。很多工具只负责记录，不负责将记录转化为统计图表或复盘材料，长期使用价值有限。

LifeButler 的设计目标是：

1. 建立本地生活管理闭环：从目标到任务，从今日执行到周报复盘。
2. 降低记录成本：通过快速收集入口识别任务、账单、运动和备忘录。
3. 提升执行可见性：用今日四象限、番茄钟和首页驾驶舱展示当天重点。
4. 保证数据安全：使用 SQLite 本地数据库，避免真实数据上传到服务器。
5. 保持工程可维护性：采用模块化 PyQt6 架构，数据库迁移、业务服务层和测试同步维护。

三、系统总体架构

LifeButler 采用典型的本地桌面应用架构：

- 1. 界面层：PyQt6 实现主窗口、导航栏、业务页面、弹窗和表格交互。
- 2. 数据层：SQLite
保存所有业务数据，包括记账、任务、目标、KR、项目、周计划、健康记录、备忘录和番茄钟 session。
- 3. 服务层：将目标管理、周计划、快速收集和首页概览等逻辑封装为服务，避免 UI 直接散落复杂业务规则。
- 4. 可视化层：使用统一的 ChartWidget 封装 matplotlib 图表，保证深浅主题切换后图表配色同步刷新。
- 5. CLI 层：提供 JSON 输出的命令行接口，方便脚本或 Agent 读取系统能力、今日概览、任务、目标和周报信息。
- 6. 打包层：使用 PyInstaller 构建 macOS App，并提供 Windows 便携包构建脚本和说明。

核心文件结构如下：

```
LifeButler/
■■■■ main.py                # ■■■■
■■■■ app/
■   ■■■■ application.py    # QApplication ■■■■■■■■■■
■   ■■■■ database.py       # SQLite ■■■■schema ■■■■■ API
■   ■■■■ main_window.py   # ■■■■■■■■■■
■   ■■■■ modules/         # ■■■■
■   ■■■■ services/        # ■■■■
■   ■■■■ widgets/         # ■■■■■■■■■■
■■■■ tests/                # ■■■■
■■■■ docs/                 # ■■■■AI ■■■■■■■■■■
■■■■ packaging/           # macOS ■ Windows ■■■■
■■■■ resources/icon/      # App ■■■■
■■■■ scripts/             # ■■■■■■■■■■
```

四、主要功能实现

4.1 首页驾驶舱

首页驾驶舱用于提供当天状态总览。页面集中展示快速收集入口、今日三件事、今日专注、本月财务、近 7 天运动、今日四象限摘要和生活雷达。

该页面的作用是让用户一打开应用就能看到当天最重要的信息，不需要在多个模块之间反复切换。生活雷达使用本地规则对任务过载、健康缺口、财务压力和目标停滞进行提示，不依赖 AI API。

4.2 快速收集

快速收集入口支持用自然文本快速记录生活事件。系统通过本地规则识别输入类型，例如：

- 午饭 28r 或 午饭 28元 会识别为记账记录。
- 跑步 30 分钟 会识别为运动记录。
- 明天提交报告 会识别为任务。
- 无法归类的内容可保存为备忘录。

识别后系统会打开确认弹窗，让用户在保存前检查和调整字段，避免错误数据直接写入数据库。

4.3 记账管理

记账模块支持新增、编辑、删除收支记录，并按月展示收入、支出和结余。分类统计使用 donut 图展示，用户可以切换查看收入或支出的分类占比。

该模块的重点不是复杂财务分析，而是为个人日常账单提供轻量记录和可视化反馈。金额、日期、类型、分类和备注都会写入本地 SQLite 数据库。

4.4 目标管理

目标管理模块包括 Objective、KR 和 Project 三层结构：

- Objective：阶段目标。
- KR：关键结果，用当前值、目标值和单位表示推进情况。
- Project：目标下的推进对象。

目标、KR、项目都支持新增、详情查看、编辑和删除。表格采用紧凑布局，避免横向滚动。KR 使用当前值/目标值 单位 的方式展示核心指标，完整信息通过详情窗口查看。

根据后期用户反馈，任务与项目之间的绑定关系已彻底移除。Project 只作为目标管理中的推进对象存在，待办任务只属于任务池、今日四象限和周计划体系。数据库 schema v5 中 todos 表不再包含 project_id 字段。

4.5 待办事项与今日四象限

待办模块包含任务池和今日四象限两个部分。

任务池用于维护所有待办事项，支持新增、编辑、删除、完成、恢复和筛选。任务双击进入编辑流程，不直接切换完成状态，避免误操作。

今日四象限用于安排当天任务：

- q1：重要紧急。
- q2：重要不紧急。
- q3：不重要紧急。
- q4：不重要不紧急。

任务通过 today_date 和 quadrant 进入今日视图。未完成任务会自动顺延显示，已完成任务不再出现在今日四象限中。

4.6 番茄钟

番茄钟可以绑定今日任务，支持开始、暂停、继续和结束。完成一次专注后，系统会写入 pomodoro_sessions 表，记录任务、开始时间、结束时间、计划分钟、实际分钟和状态。

今日四象限中的任务会显示当天完成的番茄数和累计专注分钟，从而把任务执行与专注记录关联起来。

4.7 健康记录

健康模块支持体重和运动记录：

- 体重记录支持新增、编辑、删除，并自动计算 BMI。
- 运动记录支持记录运动类型、时长和日期。
- 图表展示体重趋势和本周运动分布。

健康模块用于补充生活管理中的身体状态反馈，使系统不局限于任务和账单。

4.8 备忘录

备忘录模块用于保存非结构化信息，支持新增、编辑、删除、分类、搜索、置顶和 Markdown 预览。它承担的是“收纳零散想法”的角色，与任务、目标等结构化模块形成互补。

4.9 周计划与周报复盘

周计划模块支持把任务加入本周，设置完成度、进展备注，并标记为今日任务。周报复盘支持填写三句复盘，并导出 Markdown 文件。

这一模块让系统从“当天任务管理”扩展到“每周复盘”，使任务执行结果能够沉淀为文字总结。

4.10 设置、备份与主题

设置页面提供数据库路径查看、数据库备份和备份目录打开功能。数据库路径使用只读多行文本框显示，避免长路径被截断。

应用支持深色和浅色主题切换。主题切换由 `MainWindow._apply_theme()` 统一传播，并调用 `ChartWidget.set_theme()` 刷新图表，使 Qt 控件和 matplotlib 图表保持一致。

五、数据库设计与迁移

LifeButler 使用 SQLite 作为本地数据库。数据库由 `DatabaseManager` 统一管理，当前目标 schema version 为 5。

主要版本演进如下：

1. version 1：基础表结构，包括记账、待办、健康和备忘录。
2. version 2：增加今日四象限任务字段和 `pomodoro_sessions`。
3. version 3：兼容旧预算表约束迁移，预算界面后续移除。
4. version 4：增加目标、KR、项目、周计划和周复盘表。
5. version 5：移除任务与项目绑定字段，`todos` 不再包含 `project_id`。

数据库迁移的关键设计是：

1. 启动时读取 schema_meta.schema_version。
2. 如果当前数据库版本低于目标版本，先自动备份。
3. 按版本顺序执行 forward migration。
4. 迁移完成后更新 schema version。

这样可以避免只修改 CREATE TABLE IF NOT EXISTS 导致旧数据库缺字段、缺表或约束不一致的问题。

六、界面与交互设计

LifeButler 的界面采用左侧导航栏和右侧内容区布局。不同模块通过导航栏切换，整体风格保持一致。

本项目在开发后期针对用户反馈进行了多轮 UI 修复：

1. 表格隐藏内部 ID 列，使用 Qt.UserRole 保存数据 ID，避免表头残缺和横向滚动。
2. 目标、KR、项目表格选中互斥，点击空白处可清除蓝色选中态。
3. 删除顶部大量“编辑/删除/刷新”按钮，改为双击行打开详情，在详情窗口内编辑或删除。
4. KR 和 Project 表格改为紧凑列，减少横向滚动。
5. 设置页长路径改为多行只读文本框，确保完整可读。
6. 任务池双击改为编辑任务，不再直接切换完成状态。

这些改动提升了应用的可理解性和演示稳定性。

七、CLI 与 Agent 支持

项目提供最小 JSON CLI，入口为：

```
.venv/bin/python lifebutler system capabilities --json
```

当前 CLI 支持：

- system capabilities
- today
- task list
- task quick-capture
- goal list
- review week

所有面向 Agent 的输出都使用 JSON，成功返回包含 ok、data、meta，失败返回包含 error.code 和 error.message。写操作建议先使用 --dry-run 查看返回结构，再正式执行。

CLI 的设计目标不是替代 GUI，而是为脚本、自动化测试和外部 Agent 提供稳定入口。

八、测试与验证

项目包含自动化测试，测试使用临时数据库和 LIFE BUTLER_DB_PATH，不会污染用户真实数据。

最终提交前执行过以下验证：

```
.venv/bin/python -m py_compile main.py app/*.py app/modules/*.py app/widgets/*.py app/services/*.py app/c
.venv/bin/python -m unittest discover -s tests -v
```

测试结果：

```
Ran 23 tests ... OK
```

测试覆盖范围包括：

1. 新数据库初始化和 schema version。
2. v3、v4 临时数据库迁移到 v5。
3. 快速收集规则识别。
4. CLI JSON 输出。
5. 目标、KR、项目服务层。
6. 首页驾驶舱、周计划、设置页等 UI 构造。
7. 任务池与今日四象限同步。
8. 截图脚本可执行性。

九、打包与发布

项目已完成 macOS App 打包，并通过 GitHub Release 发布。

仓库地址：

```
https://github.com/shilinchuan/LifeButler
```

Release 地址：

```
https://github.com/shilinchuan/LifeButler/releases/tag/v0.3.0-final
```

macOS App 压缩包：

```
LifeButler-macOS-arm64-20260621.zip
```

打包过程使用 PyInstaller，App 图标采用自制的浅色圆角卡片加勾选元素。打包版默认数据库路径为：

```
~/Library/Application Support/LifeButler/data/lifebutler.db
```

这样可以避免应用把数据库写入 .app 包内部。

Windows 版本没有在 macOS 上伪造 exe，而是提供了 Windows 便携包构建脚本和说明，需在 Windows 环境中实际构建。

十、版本控制与数据安全

GitHub 仓库提交了源码、测试、文档、图标、CLI 和打包脚本。以下内容没有进入 Git 历史：

1. .venv/ 本地 Python 虚拟环境。
2. data/*.db* 本地 SQLite 数据库和 WAL/SHM 文件。
3. dist/ 打包产物。
4. outputs/ 导出报告和截图。
5. __pycache__/, .DS_Store 等缓存文件。

这保证了仓库可以公开提交，同时不会泄露本地真实数据，也不会因为打包产物导致仓库过大。

十一、实际开发中的调整

项目开发过程中，根据实际效果和用户反馈进行了几项重要调整：

1. 预算功能从界面中移除。原因是当前课程项目更聚焦任务执行、目标推进和生活反馈，预算功能容易增加复杂度。
2. 任务和项目解耦。Project 保留在目标管理体系中，任务只属于待办和周计划体系，降低用户理解成本。
3. 目标管理交互从顶部按钮改为详情窗口。双击行查看详情，在详情中编辑和删除，更符合桌面应用习惯。
4. 快速收集使用本地规则，不接入 AI API。这样可以保证稳定、可解释和可离线运行。
5. App 图标经过多轮调整，最终采用全画布浅色圆角卡片和居中勾选元素，避免桌面显示时图标缩小。

十二、项目总结

LifeButler 最终形成了一个功能较完整的本地生活管理桌面应用。它覆盖了个人生活管理中常见的任务、目标、记账、健康、备忘录和复盘场景，并通过首页驾驶舱和快速收集降低了使用门槛。

从工程角度看，项目实现了 PyQt6 模块化界面、SQLite schema migration、统一图表组件、深浅主题联动、自动化测试、CLI JSON 接口和 PyInstaller 打包。相比早期原型，最终版本已经具备较好的可演示性、可维护性和可扩展性。

后续如果继续迭代，可以重点增强以下方向：

1. 增加更完整的每日、每周、每月复盘统计。
2. 为番茄钟增加长休息、多轮循环和专注趋势图。
3. 增加更多数据导出格式。
4. 完成 Windows 便携包实际构建和测试。
5. 根据真实用户测试继续简化交互路径。

总体而言，LifeButler 已达到课程最终提交所需的完整度：源码已公开上传至 GitHub，macOS App 已通过 Release 发布，核心功能和测试验证材料完整。